

Word Count Example in Spark

The following slides show how to implement the MapReduce Word count algorithm, both in **Java** and **Python**

To understand the code, refer to

- [Introduction to Programming in Spark](#) (*for this course*)
- [Spark RDD Programming guide](#).
- [Spark Java API](#).
- [Spark Python API](#).

All links and the complete code (**WordCountExample.Java** and **WordCountExample.py**) can be found in the course Moodle.

OUTLINE

- ① Functional programming
- ② Implementation of Word Count in Spark
 - ① Spark Configuration and Input Reading
 - ② Initial Map Phase
 - ③ 1-Round Algorithm
 - ④ 2-Round Algorithm

Functional programming

Functional programming

One of the core ideas of functional programming is that **functions can be arguments to other functions**.

- In a MapReduce algorithm, the map and reduce functions used in each round can be regarded as "arguments" for the round.
- Spark's API relies heavily on passing functions. For example, many methods (e.g., RDD operations) require functions as arguments.

In Spark there are essentially **2 ways to pass functions as arguments** to a method.

- as **anonymous functions** or **lambdas** which are defined inline without a name, where the actual argument is expected.
- as **named functions** defined outside the method.

Functional programming: Java example

Let `values` be an `RDD of doubles` (i.e., of type `JavaRDD<Double>`)

USE of ANONYMOUS FUNCTIONS

- **Single statement:**

```
JavaRDD<Double> squaredValues = values.map((x) -> x*x);
```

- **Multiple statements:**

```
double fixed = 1.5;  
JavaRDD<Double> normValues = values.map((x) -> {  
    double diff = fixed - x;  
    return diff;  
});
```

Obs.: `squaredVales` (resp., `normValues`) are RDDs containing the elements of `values` squared (resp., normalized).

Functional programming: Java example

USE of NAMED FUNCTIONS

Create a separate class such as:

```
class myMethods {  
    public static double mySquare(double x) {  
        return x * x;  
    }  
}
```

Then, in the class containing the main program write:

```
JavaRDD<Double> squaredValues = values.map(myMethods::mySquare);
```

Functional programming: Python example

Let `values` be an RDD of double

USE of ANONYMOUS FUNCTIONS

```
squaredValues = values.map(lambda x: x * x)
```

Obs: Python does not allow multiple statements in anonymous functions.

USE of NAMED FUNCTIONS

```
def mySquare(x):  
    return x * x
```

```
squaredValues = values.map(mySquare)
```

Obs.: As in Java, `squaredValues` is an RDD containing the elements of `values` squared. Normalized values can be computed, in a similar way.

Implementation of Word Count in Spark

Common assumptions

- **Input:** file of documents, one document per line.
- **Output:** $(w, c(w))$ pairs, with $w \equiv$ word and $c(w) \equiv$ number of occurrences of w in the documents
- **Command Line Arguments:**
 - L : number of partitions for the RDD
 - `data_path`: where the input file is located

Spark Configuration and Input Reading: Java

```
import ....

public class WordCountExample{

    public static void main(String[] args) throws IOException {

        // SPARK SETUP
        SparkConf conf = new SparkConf(true).setAppName("WordCount");
        JavaSparkContext sc = new JavaSparkContext(conf);

        // INPUT READING
        int L = Integer.parseInt(args[0]);
        JavaRDD<String> docs = sc.textFile(args[1]).repartition(L).cache();
```

Spark Configuration and Input Reading: Python

```
def main():  
  
    # SPARK SETUP  
    conf = SparkConf(true).setAppName('WordCount')  
    sc = SparkContext(conf=conf)  
  
    # INPUT READING  
    L = sys.argv[1]  
    L = int(L)  
    data_path = sys.argv[2]  
    docs = sc.textFile(data_path).repartition(L).cache()
```

- **Spark Setup**: sets the configuration object, the context object, and the name of the application. (See “Introduction to Programming in Spark” for details)
- **textfile**: transforms the input file into an RDD of Strings, whose elements correspond to the distinct lines of the file (i.e., the documents in the example)
- **repartition(L)**: subdivides the RDD elements, **at random**, into *L* groups, referred to as *partitions*
- **cache()**: tells Spark that the first time that the RDD is materialized, its elements must be stored permanently in the executors’ main memories, if they fit.

Initial Map Phase

RDD of Strings (1 String $\equiv$ 1 Documents)

↓ flatMapToPair (Java) or flatMap (Python)

RDD of $(w, c(w))$ pairs

At this point $\forall$ word w we have a pair $(w, c(w))$ for every document where w occurs, and the value $c(w)$ is the number of occurrences of w in that particular document

Initial Map Phase: Java

```
JavaPairRDD<String, Long> wordCounts = docs
    .flatMapToPair(myMethods::wordCountPerDoc) // <-- MAP PHASE
    ....

Class myMethods {
    public static Iterator<Tuple2<String,Long>>
        wordCountPerDoc(String document) {
        String[] tokens = document.split(" ");
        HashMap<String, Long> counts = new HashMap<>();
        ArrayList<Tuple2<String, Long>> pairs = new ArrayList<>();
        for (String token : tokens) {
            counts.put(token, 1L + counts.getDefault(token, 0L));
        }
        for (Map.Entry<String, Long> e : counts.entrySet()) {
            pairs.add(new Tuple2<>(e.getKey(), e.getValue()));
        }
        return pairs.iterator();
    }
    ....
}
```

- **Tuple2**: In Java, key-value pairs are represented using type `Tuple2<K,V>`, where `K` and `V` are the domains for keys and values, respectively. For a `Tuple2` object `T`
 - `T._1()` returns the key
 - `T._2()` returns the value
- **JavaPairRDD**: In Java, **RDDs of (key,value) pairs** are represented as instances of type `JavaPairRDD<K,V>`, whose elements are instances of type `Tuple2<K,V>`.
- **flatMapToPair**: transforms each RDD element (a document, in the example) into ≥ 0 pairs (of type `Tuple2`). The function passed as argument (`wordCountPerDoc`, in the example) must return the pairs as an **iterator**.

Obs. In case exactly one pair must be returned, **MapToPair** instead of **flatMapToPair** can be used.

Initial Map Phase: Python

```
def word_count_per_doc(document):  
    pairs_dict = {}  
    for word in document.split(' '):  
        if word not in pairs_dict.keys():  
            pairs_dict[word] = 1  
        else:  
            pairs_dict[word] += 1  
    return [(key, pairs_dict[key]) for key in pairs_dict.keys()]  
  
def word_count_1(docs):  
    word_count = (docs.flatMap(word_count_per_doc) # <-- MAP PHASE  
    ....
```

In Python, `flatMapToPair` does not exist, and we use `flatMap` instead. (Python does not care to know whether the output are pairs or not)

- In Python, **key-value pairs** are represented using built-in **Python tuples**
- **flatMap**: transforms each RDD element (a document, in the example) into ≥ 0 new elements, through the function passed as argument (`word_count_per_doc`, in the example).

Obs. In case exactly one pair must be returned, **map** instead of **flatMap** can be used.

1-Round Algorithm: Java

```
JavaPairRDD<String, Long> wordCounts = docs
    .flatMapToPair(myMethods::wordCountPerDoc) // <-- MAP PHASE
    .groupByKey() // SHUFFLING+GROUPING
    .mapValues((it) -> { // <-- REDUCE PHASE
        long sum = 0;
        for (long c : it) sum += c;
        return sum;
    })
```

Observation: to illustrate the various possibilities to pass functions, `flatMapToPair` receives in input a named function, while `mapValues` receives in input an anonymous function

1-Round Algorithm: Python

```
word_count = (docs
    .flatMap(word_count_per_doc) # <-- MAP PHASE
    .groupByKey()                # <-- SHUFFLE+GROUPING
    .mapValues(lambda vals: sum(vals))) # <-- REDUCE PHASE
```

RDD of Strings (1 string $\equiv$ 1 document)

↓ flatMapToPair

RDD of $(w, c(w))$

{ in $(w, c(w))$ the count $c(w)$ is
relative to one document

↓ groupByKey

RDD of $(w, \text{list-of-counts})$ pairs

{ the list contains all
counts from the various
documents

↓ mapValues

RDD of $(w, c(w))$ pairs

{ the $c(w)$'s are the
final counts

- **groupByKey**: collects the elements (pairs) of the RDD with the same key, and **returns an RDD of (key,list-of-values) pairs**. In Java, list-of-values is returned as an iterator.
- **mapValues**: transforms each pair by retaining the key and processing its value through the function passed as argument. In the example, the value of each pair is, in fact, a list (provided as an iterator in Java).

More Efficient Implementation

The combination of `groupByKey` and `mapValues` can be replaced by the ready-made method `reduceByKey` as follows:

JAVA:

```
JavaPairRDD<String, Long> wordCounts = docs
    .flatMapToPair(myMethods::wordCountPerDoc) // <-- MAP PHASE
    .reduceByKey((x, y) -> x+y);           // <-- REDUCE PHASE
```

PYTHON:

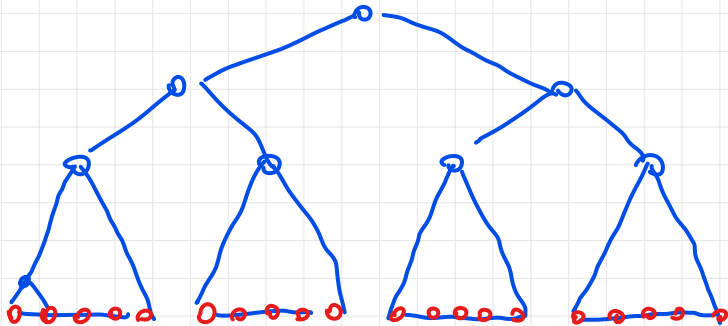
```
word_count = (docs.flatMap(word_count_per_doc) # <-- MAP PHASE
    .reduceByKey(lambda x, y: x + y))          # <-- REDUCE PHASE
```

- `reduceByKey`: for each key k occurring in the RDD, the method combines the values of all pairs with key k in the RDD, applying the function passed as argument, which is regarded as a binary operator (the sum, in the example), in a tree-like fashion.

IMPORTANT OBSERVATIONS:

- The programmer has no control on the sequence of binary aggregations performed by `reduceByKey`, hence the function passed as argument must be commutative and associative.
- For efficiency, Spark will do the aggregation first within the partitions of the RDD, and then across partitions. Thus, it is a sort of 2-round algorithm in disguise

Reduce By Key aggregates values using a binary-tree schedule like this



Leaves $\equiv$ values to be aggregated

The aggregation starts from distinct partitions (bottom subtrees)

2-Round Algorithm: general structure

Round 1

- **Map Phase:** for each document D_i compute local count $c_i(w)$ of each word w and partition $(w, c_i(w))$ pairs among L partitions.
- **Reduce Phase:** for each word w aggregate local counts separately for each partition producing (at most) L pairs $(w, c(x, w))$, one for every partition x , with $0 \leq x < L$

Round 2

- **Map Phase:** empty.
- **Reduce Phase:** for each word w aggregate the $c(x, w)$ counts into the final count $c(w)$ and produce the pair $(w, c(w))$.

We will explore **two partitioning techniques**:

- ① Random keys assigned on the fly during groupBy in Round 1
- ② Reuse of the RDD partitions defined by Spark

NOTE. In the complete `WordCountExample` code that we provide, you will find a third version where random keys are assigned explicitly during the Map Phase of Round 1. This version, which is not discussed in these slides, is functionally equivalent (but less efficient) than the one that does the key assignment within groupBy. However, it is useful as a further example of how a Map Phase can transform an initial set of elements.

2-Round Algorithm with Random Keys: Java

Assume that #partitions L is known (e.g., read as command-line argument)

```
JavaPairRDD<String, Long> wordCounts = docs
    .flatMapToPair(myMethods::wordCountPerDoc) <-- MAP PHASE (R1)
    .groupByKey((wordcountpair) -> randomGenerator.nextInt(L))
    // IMPLEMENTS RANDOM KEY ASSIGNMENT, SHUFFLE AND GROUPING

    .flatMapToPair(myMethods::gatherPairs) <-- REDUCE PHASE (R1)
    .reduceByKey((x, y) -> x+y); // <-- REDUCE PHASE (R2)
```

RDD of Strings (1 string $\equiv$ 1 document)

↓ flatMapToPair

RDD of $(w, c(w))$ pairs { $\forall w$ we have at most
one $c(w)$ per doc

↓ groupByKey + random assignment

RDD of $(k, \text{list of } (w, c(w)) \text{ pairs})$

→ pairs assigned to
key $k \in [0, L)$

↓ flatMapToPair (gather - pairs)

RDD of $(w, c(w))$ pairs



reduce By Key

at this point we
have $\forall w$ at most
one count $c(w)$ for
every partition $\Rightarrow$
 $\leq L$ counts per word

RDD of $(w, c(w))$ pairs

$\rightarrow$ final counts

```

class myMethods {

    public static Iterator<Tuple2<String,Long>>
        wordCountPerDoc(String document) {... seen before ...}

    public static Iterator<Tuple2<String,Long>> gatherPairs
        (Tuple2<Integer,Iterable<Tuple2<String, Long>>> element) {
        HashMap<String, Long> counts = new HashMap<>();
        for (Tuple2<String, Long> c : element._2()) {
            counts.put(c._1(),c._2()+counts.getOrDefault(c._1(),0L));
        }
        ArrayList<Tuple2<String, Long>> pairs = new ArrayList<>();
        for (Map.Entry<String, Long> e : counts.entrySet()) {
            pairs.add(new Tuple2<>(e.getKey(), e.getValue()));
        }
        return pairs.iterator();
    }
}

```

`groupBy` does the following actions:

- Transforms each element of the RDD into a pair (key,element), where key is assigned through the function passed as argument. In the example, a random integer key in $[0, L)$ is assigned, and element is a (word,count) pair.
- For each key k , collects the pairs with key k into a new pair (k , list-of-values). In the example, the list-of-values is a list of (word,count) pairs.

Observation. Thus, in the example, `groupBy` produces, as a result, an RDD of (key, list-of-(word,count)-pairs). The elements of this RDD are then processed, individually, by the subsequent `flatMapToPair` (using the `gatherPairs` function in the example)

2-Round Algorithm with Random Keys: Python

```
def gather_pairs(pairs):
    pairs_dict = {}
    for p in pairs[1]:
        word, occurrences = p[0], p[1]
        if word not in pairs_dict.keys():
            pairs_dict[word] = occurrences
        else:
            pairs_dict[word] += occurrences
    return [(key, pairs_dict[key]) for key in pairs_dict.keys()]

def word_count_2(docs, L):
    word_count = (docs
        .flatMap(word_count_per_doc)                # <-- MAP PHASE (R1)
        .groupBy(lambda x: (rand.randint(0,L-1)))    # <-- SHUFFLE+GROUPING
        .flatMap(gather_pairs)                       # <-- REDUCE PHASE (R1)
        .reduceByKey(lambda x, y: x + y))            # <-- REDUCE PHASE (R2)
    return word_count
```

The sequence of RDDs created by these
transformations are the same as those
for Jane (see Slides 29-30)

2-Round Algorithm with Spark Partitions: Java

```
JavaPairRDD<String, Long> wordCounts = docs
    .flatMapToPair(myMethods::wordCountPerDoc)
    .mapPartitionsToPair((element) -> {    // <-- REDUCE PHASE (R1)
        HashMap<String, Long> counts = new HashMap<>();
        while (element.hasNext()){
            Tuple2<String, Long> tuple = element.next();
            counts.put(tuple._1(), tuple._2() +
                        counts.getDefault(tuple._1(), 0L));
        }
        ArrayList<Tuple2<String, Long>> pairs = new ArrayList<>();
        for (Map.Entry<String, Long> e : counts.entrySet()) {
            pairs.add(new Tuple2<>(e.getKey(), e.getValue()));
        }
        return pairs.iterator();
    })
    .groupByKey()    // <-- SHUFFLE+GROUPING
    .mapValues(.. as before ..);
```

↳ this can be substituted by reduceByKey

RDD of Strings

↓ flatMapToPair

RDD of $(w, c(w))$ pairs $\begin{matrix} \nwarrow w \\ \text{(one count per doc)} \end{matrix}$

↓ mapPartitionsToPair

RDD of $(w, c(w))$ pairs

↓ groupByKey + mapValues

RDD of $(w, c(w))$ pairs
→ final count

Now for every word w we have at most L counts, 1 per partition

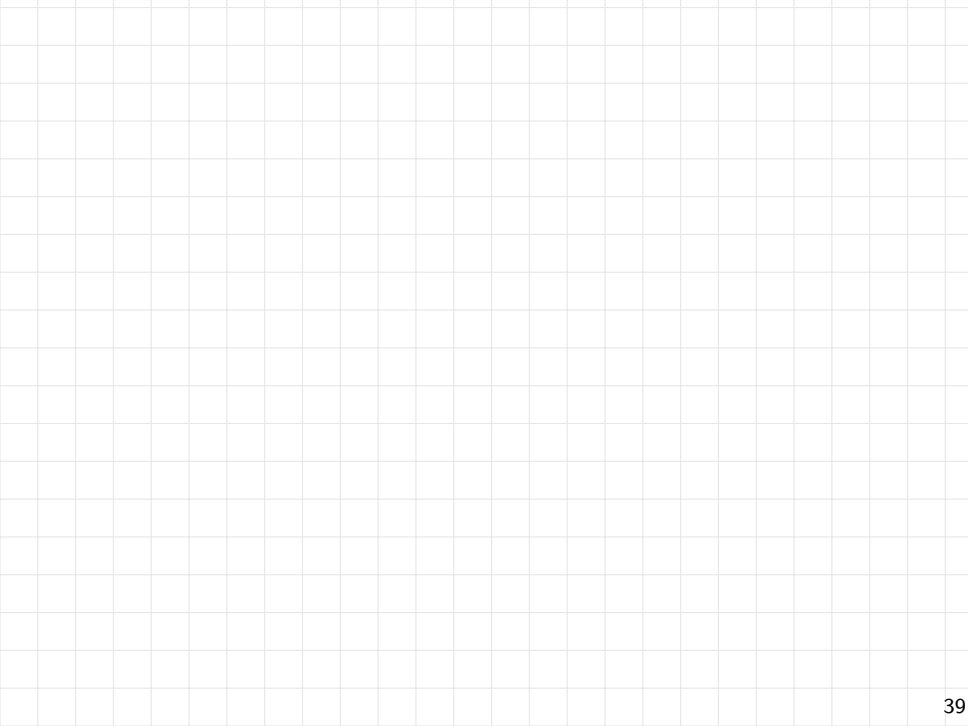
For each partition of the RDD (created by Spark), separately, `mapPartitionsToPair` applies the function, passed as argument, to the elements of RDD which belong to that partition. The elements of the partition are provided in input to the function as an iterator. The function must return ≥ 0 pairs (of type `Tuple2`)

In the example, each partition contains (word,count) pairs, where the counts are relative top individual documents. The execution `mapPartitionsToPair` will result again in an RDD of (word,count) pairs where, however, the counts have been aggregated within each partitions.

2-Round Algorithm with Spark Partitions: Python

```
def gather_pairs_partitions(pairs):
    pairs_dict = {}
    for p in pairs:
        word, occurrences = p[0], p[1]
        if word not in pairs_dict.keys():
            pairs_dict[word] = occurrences
        else:
            pairs_dict[word] += occurrences
    return [(key, pairs_dict[key]) for key in pairs_dict.keys()]

def word_count_with_partition(docs):
    word_count = (docs.flatMap(word_count_per_doc) # <-- MAP PHASE (R1)
        .mapPartitions(gather_pairs_partitions) # <-- REDUCE PHASE (R1)
        .groupByKey() # <-- SHUFFLE+GROUPING
        .mapValues(lambda vals: sum(vals))) # <-- REDUCE PHASE (R2)
    return word_count
```



IMPORTANT OBSERVATIONS. If an RDD is the result of a sequence of transformations:

- Its `partitioning` is inherited from the one produced by the last invocation of the `repartition` method, if any, or from the one initially created when data where loaded in memory.
- However, since data may have evolved substantially, the balancing is not guaranteed. To ensure it, the `repartition` method should be invoked, which, however, involves shuffling the data.